

Daffodil Firmware — Wally 18 Panel Sensor Integration

Changes made 2026-07-23 · written, not yet flashed PENDING UPLOAD

Why this happened: Wally Build 18 adds a USB/panel current sensor (INA219 at I2C address `0x45`), confirmed working on real hardware via Adafruit's stock example sketch. This session integrated that sensor into Daffodil's firmware end-to-end: reading it, storing it, using it for cloud detection, and exposing it as a remote diagnostic — plus a couple of unrelated bugs found and fixed along the way.

1. Struct changes — `DigitalStablesData.h`

`DigitalStablesData` was already near its practical LoRa size ceiling, so instead of growing it, space was freed by removing a field that was always redundant and shrinking two oversized buffers.

Field	Before	After	Why
<code>sensor1name</code> / <code>sensor2name</code>	<code>char[8]</code> each	<code>char[6]</code> each	Freed 4 bytes total for the new current field
<code>scepticAvailablePercentage</code>	<code>float</code> (4 bytes)	removed → replaced by <code>panelVoltage</code>	Was purely derived on-device from <code>measuredHeight</code> and a firmware constant — never needed on the wire. Its only consumer (an LED color block) now recomputes it locally.
<code>panelVoltage</code>	—	<code>float</code> , default <code>-99</code>	Wally 0x45 bus voltage. <code>-99</code> sentinel when sensor absent (matches <code>batteryCurrent</code> 's existing convention).
<code>panelCurrent</code>	—	<code>float</code> , default <code>-99</code> (appended at struct end)	Wally 0x45 current in mA. Same -99-if-absent convention.

Net `sizeof(DigitalStablesData)` is **unchanged** — verified with a throwaway struct-layout replication (g++, matching field order/types) before committing to the change. LoRa's size-based packet dispatch (`packetSize == sizeof(DigitalStablesData)`) is unaffected.

Byte layout shifted. Every field *after* `sensor1name` / `sensor2name` moved by `-4` bytes in the raw struct layout. Any receiver that mirrors this struct byte-for-byte (not just by size) needs the identical updated

struct definition.

2. New sensor wiring — `Daffodil.ino`

- New `Adafruit_INA219 solarIna219(0x45)` object + `foundINA219Solar` flag, detected during the existing I2C scan.
- Confirmed via KiCad netlist trace that Wally's shunt (`R13` , part `LVK12R050CER`) is electrically identical to Daffodil's own battery-sense shunt (`R21`) — so the same hand-tuned 50mΩ calibration applies to both sensors. That calibration logic was factored out into one shared `configureINA219Calibration(addr)` helper, used for both `0x41` and `0x45` , instead of duplicating magic numbers.
- `readSensorData()` populates `panelVoltage` / `panelCurrent` every cycle when the sensor is present, falls back to `-99` when it isn't — never blocks or hangs on a missing sensor.
- **Not** added to the mandatory I2C error-reporting block — this sensor is genuinely optional right now (older Wally boards won't have it), so its absence isn't treated as a fault.

3. Cloudy detection now considers panel current

Two places in the firmware decide "is it currently cloudy" — the LED/telemetry `operatingStatus` flag, and the sleep-duration calculation in `goToSleep()` . Both already OR together a lux signal, a `V50_I` voltage signal, and a weather-forecast signal. A fourth signal was added to both:

```
bool panelCurrentSaysCloudy = foundINA219Solar
    && digitalStablesData.panelCurrent >= 0
    && digitalStablesData.panelCurrent < panelCurrentCloudyThreshold_mA;
```

This is a more direct signal than `V50_I`: current is genuinely ~0mA with no real panel output, whereas `V50_I` voltage can float upward at night from a small (µA-scale) reverse-leakage path inside the battery charger IC — a whole separate investigation from earlier this week that turned out to be a benign, documented charger IC characteristic, not a fault.

Threshold: `panelCurrentCloudyThreshold_mA = 60.0`

Reasoned from real numbers, not guessed: the system draws roughly **60mA** at baseline and spikes to ~**130mA** only briefly during LoRa transmission (not sustained load, so it doesn't belong in a threshold meant to characterize ongoing conditions). Below 60mA, the panel can't even cover baseline draw, so the battery must be net-discharging — a solid definition of "not really harvesting."

Still wants validation against real sunny-vs-cloudy field data before fully trusting it.

4. New remote diagnostic — I2C device status

Extends the on-demand diagnostics system built earlier this week (triggered via `EnableDiagnostics#<type>` / `DisableDiagnostics` over LoRa, no UART needed). A new diagnostic type reports which I2C sensors were actually detected at boot — useful for confirming remotely whether e.g. the light sensor or the new current sensor is really present, without physical access.

Bit	Device
0	LCD (0x03)
1	Temp/SHT (0x40)
2	ADS1115 (0x48)
3	BH1750 light sensor (0x23)
4	INA219 battery (0x41)
5	PCF8563T RTC (0x51)
6	INA219 solar/panel (0x45) — the new Wally 18 sensor
7	reserved (DS18B20 deliberately excluded — it's OneWire, not I2C)

Unlike the earlier TX-current diagnostic (which needs to sample live current *during* an actual LoRa transmission), I2C presence is already known right after the boot-time scan — so this diagnostic sends on the very next cycle after being enabled, with no sampling delay.

```
#define DIAGNOSTIC_TYPE_I2C_STATUS 2

struct I2CStatusDiagnosticPayload {
    uint8_t deviceFoundMask;
};
```

This was added as a second member of the existing `DiagnosticRecord`'s payload `union` (alongside the TX-current payload) — a tagged union, where the sibling `diagnosticType` field says which member is currently valid. This keeps every diagnostic kind inside one fixed-size LoRa packet shape (dispatch stays by `diagnosticType`, not by inventing a new unique packet size per diagnostic), at the cost of the small 1-byte payload still riding inside the larger ~101-byte envelope sized for the biggest diagnostic type. Acceptable since diagnostics are on-demand and rare, not periodic traffic.

5. Bugs found and fixed along the way

Buffer overflow risk in sensor name handling. Roughly 15 call sites in `setup()` called `toCharArray(digitalStablesData.sensorName, 12)` — a hardcoded length of 12 against a buffer that was only 8 bytes (now 6). Any configured flow/tank sensor name ≥ 8 characters was already silently overflowing into adjacent struct memory before this session touched anything. All call sites now use

`sizeof(...)` instead of a hardcoded number, and the serial-command path (`SetDeviceSensorConfig`) that sets these names from user input was given the same bound — previously it trusted `.length()+1` from raw serial input with no cap at all.

6. Files touched

`Daffodil.ino` `DigitalStablesData.h` `DataManager.cpp` `DaffodilWifiManager.cpp`
`DigitalStablesDataSerializer.cpp`

The three shared-library files beyond `Daffodil.ino` and `DigitalStablesData.h` weren't optional extras — they're hard compile dependencies. `DataManager.cpp` (CSV export + local web JSON), `DaffodilWifiManager.cpp` (another JSON generator), and `DigitalStablesDataSerializer.cpp` (the `GetSensorData` USB-serial text dump) all referenced the renamed `scepticAvailablePercentage` field directly — the sketch would not have compiled without updating them too. Renames were done in place; the new `panelCurrent` field was appended at the very end of each output format so no existing field's position shifted in the positional serial-text protocol.

What's not done yet

1. Flash and validate on real hardware. Nothing above has been uploaded or tested as an integrated whole yet — only the bare sensor was tested standalone via Adafruit's example sketch. Confirm `foundINA219Solar` actually detects `true` on the real board, and watch `panelVoltage / panelCurrent` for a few real cycles before trusting the cloudy-detection wiring.

2. Patch the teleonome backend. Everything downstream that parses this firmware's output needs matching updates:

- The `DigitalStablesData` binary layout — field rename, resize, and the byte-offset shift for every field after the shrunk name arrays.
- A receive-side handler for `DiagnosticRecord.diagnosticType == 2`, decoding the bitmask per the table above.
- The `GetSensorData` positional serial-text protocol — one renamed field, one new field appended at the end.

3. `data/index.js` in the separate `daffodilwebapp` project still reads

`currentData.scepticAvailablePercentage` — not fixed here, since this repo's `data/` directory is just a synced copy (`updateData.sh`) and editing it in place would be overwritten on the next sync. Needs the rename in the actual source project, plus consider surfacing `panelCurrent` there too.

4. Unrelated pre-existing bug, still open. `Daffodil.ino` checks

`digitalStablesData.wifiStatus==2` , but `getWifiStatus()` in the shared library returns a plain `bool` — and even a corrected enum comparison should check `==3` per the struct's own documentation comment (`2=STA no internet, 3=STA+internet`). Flagged during the original pre-flash review, not part of this session's work, but worth fixing while already in this file for the upload.